

Day: 1 Task

What I Understand About the Product Manager Role

A Product Manager is the person who owns the "what" and the "why" of a product. While engineers figure out "how to build it" and designers figure out "how it should look," the PM answers a more fundamental question — "should we build this at all, and for whom?"

On any given day, a PM is talking to users to understand their problems, writing requirements for the engineering team, prioritising which features matter most, reviewing data to see if the last release actually worked, and aligning everyone like design, tech, business — around a shared vision. They are the backbone of product team.

What a PM does not do is important to understand as well. We don't write the code, design the screens, or manage the project timeline. We don't sit above the team making unilateral decisions. PM leads through influence, not by any authority or Dominance. If engineering team disagrees with a decision, the PM cannot simply "order" them. We have to convince, collaborate, and earn trust.

Why does this role matter so much? Because without someone owning the vision and the user's voice, teams build things efficiently that sometimes nobody needs. A PM is the reason the product solves a real problem, reaches the right users, and keeps improving after launch. They are the bridge between what the business wants, what the user needs, and what technology can do and finding that intersection is one of the hardest and most valuable things in any company.

Skills mentioned in the job description:

Responsibilities-based skills:

- Product vision, strategy, and roadmap definition (✓)
- Requirement gathering and prioritisation (✓)
- Stakeholder management (✗)
- Cross-functional team collaboration (✓)
- Product lifecycle management (✓)
- Product performance analysis (✓)
- User feedback analysis (✓)
- Market trend analysis (✗)
- Sprint planning (✗)
- Backlog management (✓)
- Agile execution (✓)

Required Qualifications-based skills:

- Product lifecycle management (✓)
- Agile methodologies (✓)
- Stakeholder management (✗)
- Cross-functional collaboration (✓)
- Jira (✓)
- Confluence (✗)
- Figma (✗)
- Analytical thinking (✓)
- Problem-solving (✓)
- Decision-making (✓)

Short Note: Waterfall vs Agile vs Hybrid

- Waterfall is the oldest and simplest approach. You plan everything upfront, then execute phase by phase, requirements first, then design, then development, then testing, then delivery. Nothing moves to the next phase until the previous one is fully done. This works perfectly when you know exactly what you're building from day one and nothing will change. Think construction, government projects, or legal compliance systems. The risk is that the client only sees the final product at the very end and by then, if something is wrong, fixing it is expensive.
- Agile flips this completely. Instead of planning everything upfront, you work in short cycles called sprints (usually 2 weeks). At the end of every sprint, you deliver a working piece of the product, get feedback, and adjust. This is ideal when requirements are uncertain or will evolve — like building an app or a new digital product. The client is involved throughout, which means you catch mistakes early and build exactly what they want, not what you thought they wanted six months ago.
- Hybrid takes the best of both. You use Waterfall's discipline for governance; fixed scope, documented requirements, approval gates but execute the actual work in Agile sprints. This suits large enterprises or regulated industries like banking where you can't just "figure it out as you go," but you still need the speed and adaptability of Agile for the delivery phase.

